



RISC-V Pipelined

Sophia Leñero Gómez

A01639462

Gregorio Alejandro Orozco Torres

A01641967

Miguel Alonso De La Rosa Zamora

A01646106

Diseño de sistemas en chip

Gpo 501

Domingo 14 de Junio 2026



dirección de la siguiente posición de memoria usando $PC+4$. Tanto la instrucción como los valores asociados al PC son almacenados en el registro de pipeline correspondiente para ser utilizados en la siguiente etapa. La segunda etapa es la de DECODE donde la instrucción es interpretada para determinar el tipo de operación a realizar. Aquí se determina qué valores son requeridos según el tipo de valor para las siguientes etapas. Una vez sabiendo justo cuáles son los operandos y señales de control, sigue la etapa de EXECUTE, en donde se llevan a cabo las operaciones aritméticas y lógicas necesarias usando la ALU. Además en esta fase se evalúan condiciones necesarias para determinar si el flujo del programa debe modificarse. Después está la etapa de MEM access, encargada de interactuar con la memoria de datos cuando la instrucción así lo requiere. En el caso de instrucciones de carga (*load*), se obtiene la información almacenada en la dirección calculada previamente por la ALU. Para instrucciones de almacenamiento (*store*), los datos provenientes del banco de registros son escritos en la posición correspondiente de memoria. Finalmente, la etapa de Write Back tiene como propósito actualizar el banco de registros con el resultado definitivo de la instrucción.

Funcionamiento de la unidad de hazard.

La unidad de *hazards* se encarga de detectar posibles *hazards* capaces de romper el flujo adecuado del procesador; estos *hazards* se categorizan en dos tipos: los de control y los de datos. Los *hazards* de control se refieren a cuando una instrucción de salto cambia el flujo que el procesador estaba ejecutando, lo que altera o se contrapone con las instrucciones que el *pipeline* tenía previamente. El otro tipo de *hazard* son los de datos, que se refieren a cuando alguna etapa necesita un dato que todavía no se encuentra disponible porque aún está siendo procesado por una instrucción anterior.

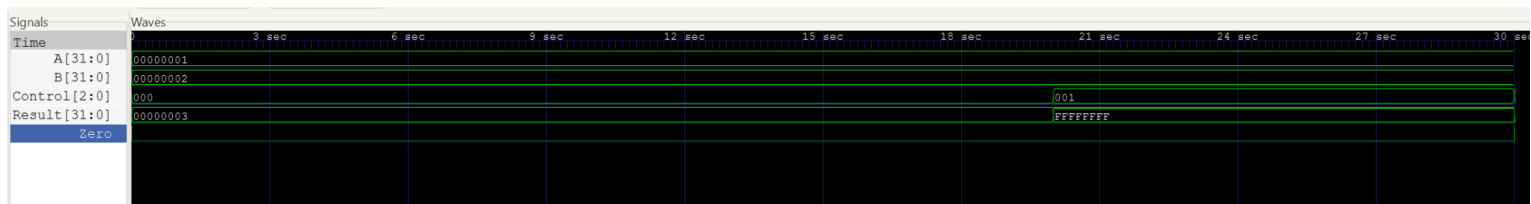
El Hazard Unit tiene el propósito de resolver estas situaciones integrando las señales de Stall, Flush y Forwarding. Básicamente, monitorea los registros fuente y los compara con los registros destino presentes en las etapas de execute, memory access y write back. En caso de detectar un hazard, se genera una señal para detener temporalmente la actualización del Program Counter y de los registros IF/ID, evitando que nuevas instrucciones ingresen al pipeline mientras se resuelve la dependencia. De igual manera, se activan las señales de limpieza correspondientes para invalidar la información contenida en determinadas etapas, insertando una burbuja (*bubble*) que permite conservar la correcta secuencia de ejecución. Por otro lado, cuando la dependencia detectada puede resolverse sin detener el procesador, la unidad habilita el mecanismo de forwarding. Este mecanismo redirige directamente los



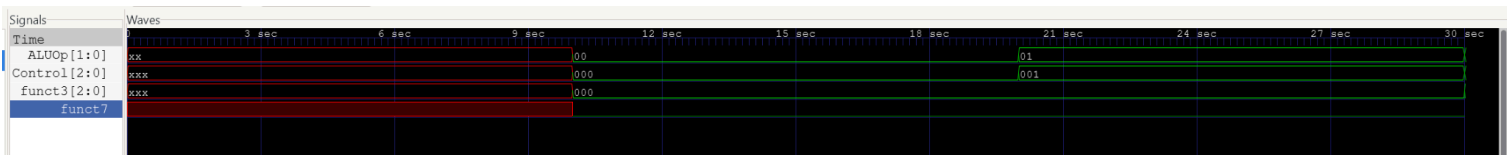
resultados obtenidos en etapas posteriores hacia la etapa de ejecución que los requiere, evitando esperar a que dichos datos sean escritos en el banco de registros. Gracias a esta estrategia, podemos reducir la cantidad de ciclos desperdiciados y mejorar el desempeño del sistema.

Capturas de simulación

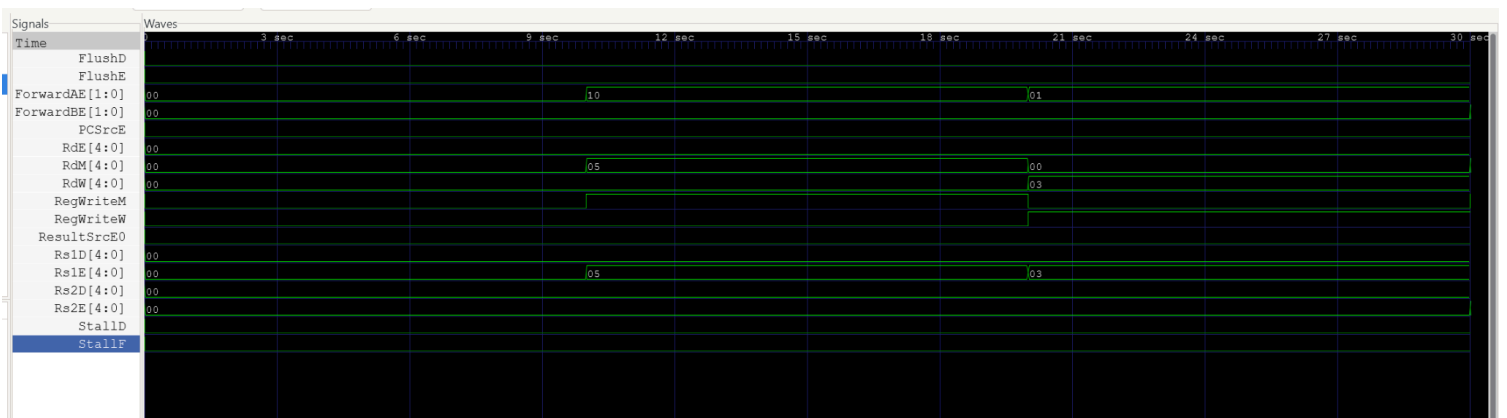
alu_tb;



alu_decoder_tb;

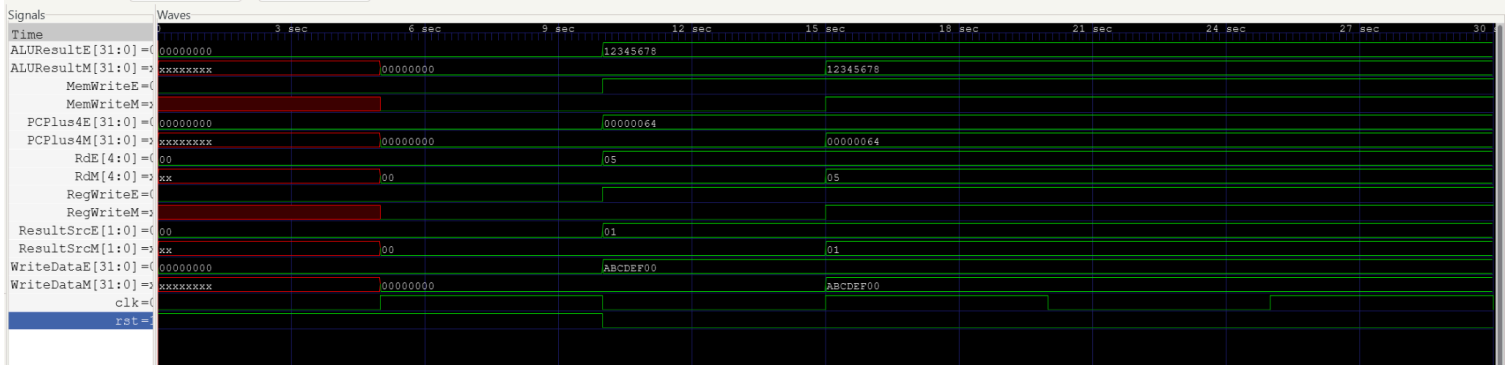


hazard_unit_tb;





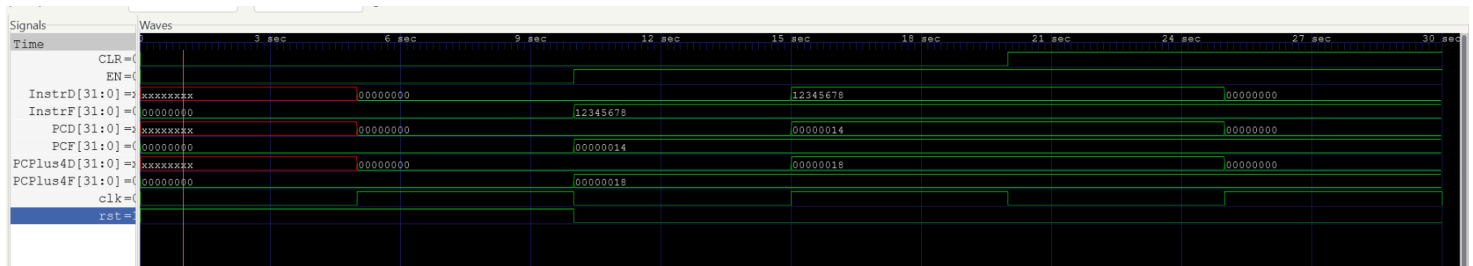
pipereg_ex_mem_tb;



pipereg_id_ex_tb;

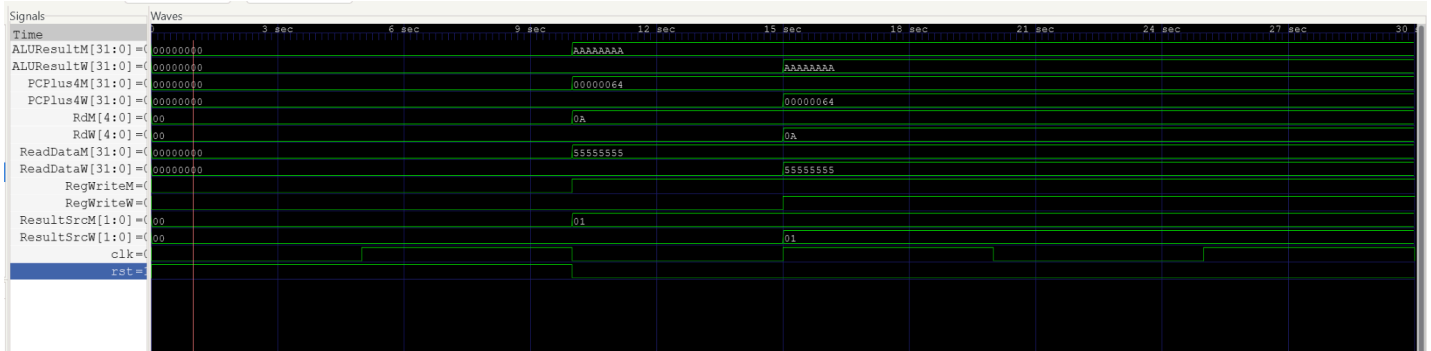


pipereg_if_id_tb;

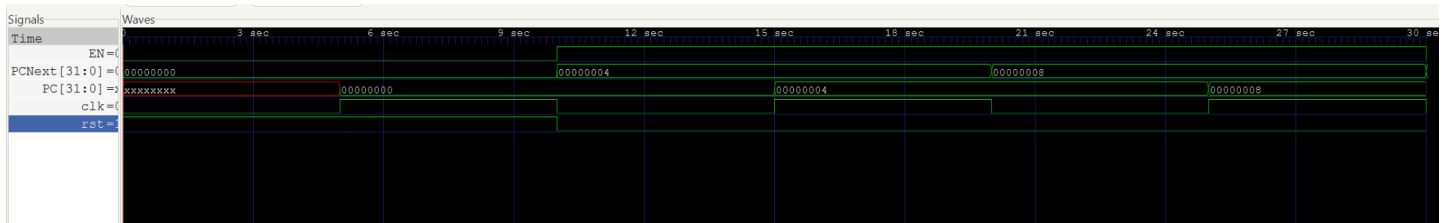




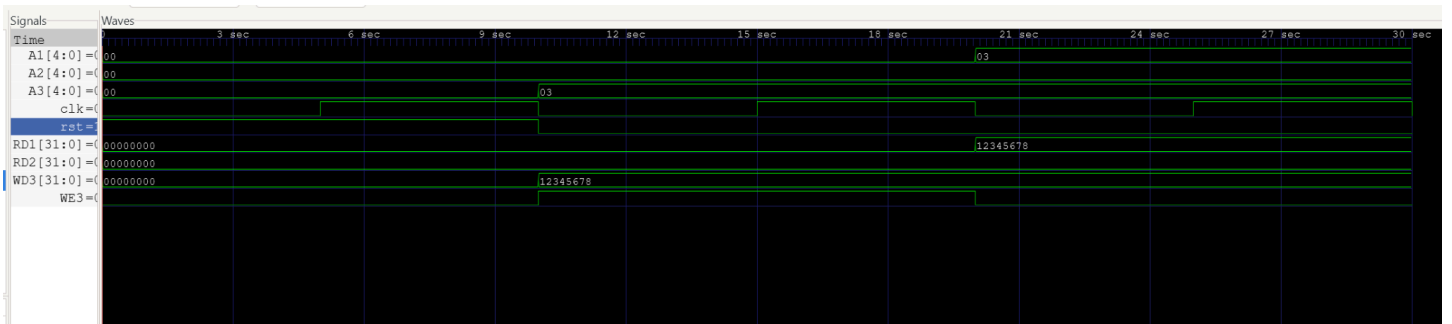
pipereg_mem_wb_tb;



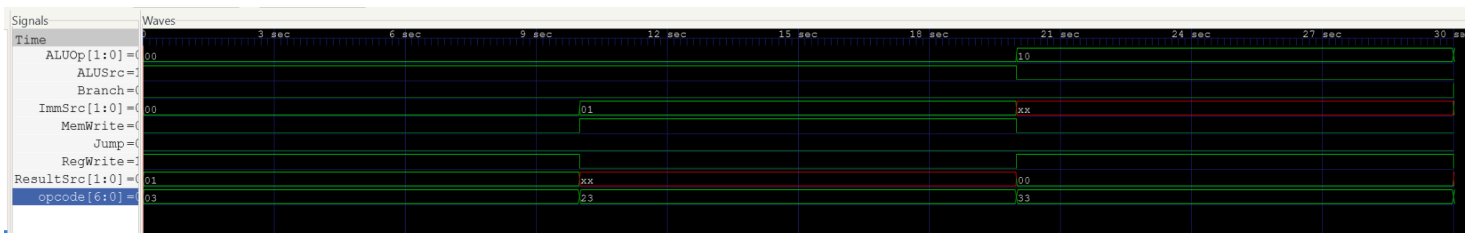
program_counter_tb;



register_file_tb;

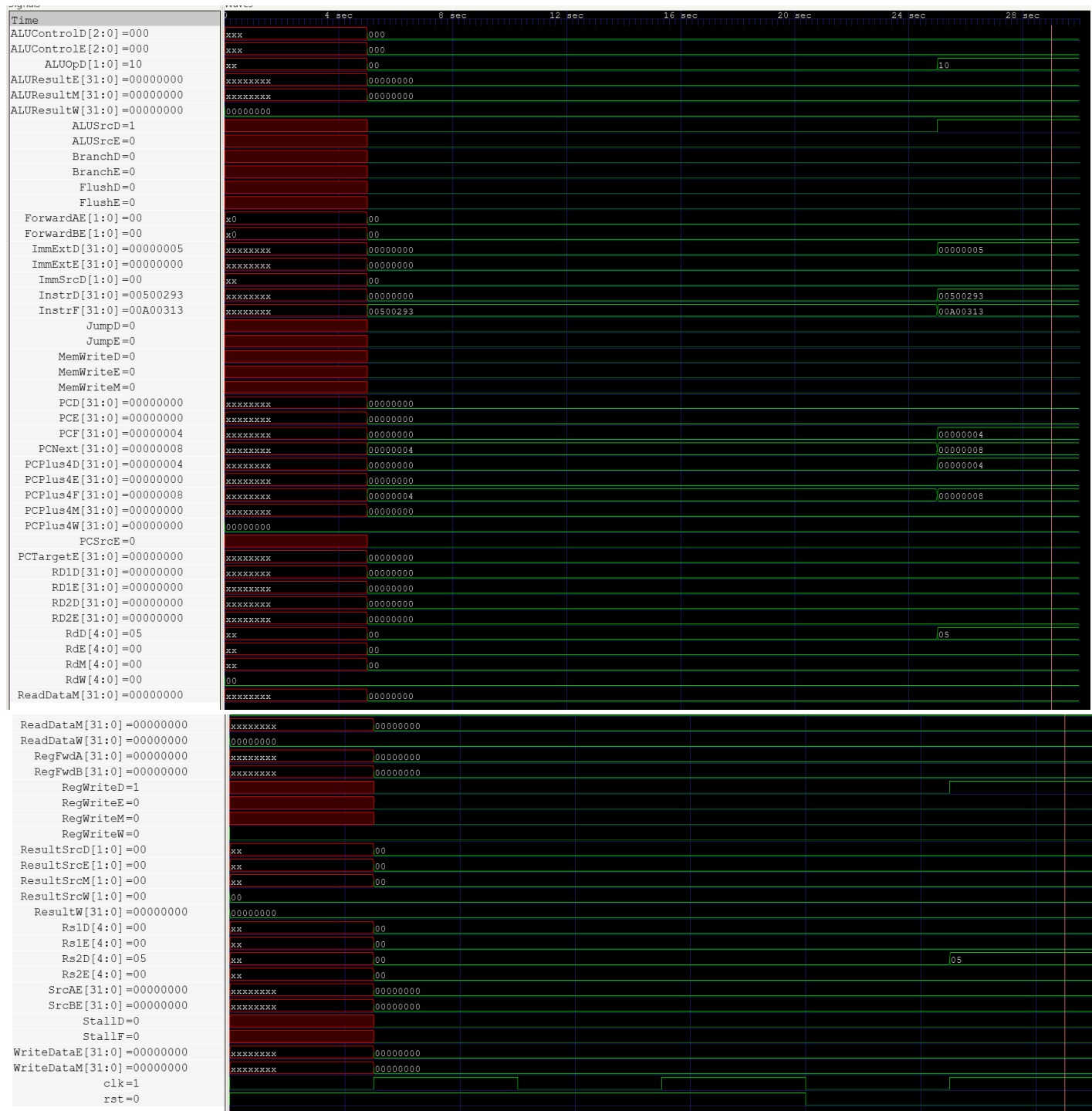


main_decoder_tb;





top_tb;





Resultados observados con testbench

Se utilizaron diversos test benches para probar la funcionalidad de cada uno de los módulos, entre los test benches, podemos ver en cada uno el correcto funcionamiento de cada bloque que compone la arquitectura de este sistema, de entre estos, hay algunos que nos dan feedback puntual y específico de nuestro sistema, los cuáles se estarán detallando a continuación.

La prueba del Hazard Unit nos permitió verificar que estuvieran funcionando bien los mecanismos de detección y resolución de dependencias de datos dentro de nuestro procesador. A partir de los estímulos que establecimos observamos que al existir una coincidencia entre los registros fuente de la etapa EXECUTE y los registros destino de las demás etapas consiguientes, las señales ForwardAE y ForwardBE adoptaron los valores esperados para habilitar forwarding. De igual manera comprobamos que se detectaban bien los casos de dependencia del tipo load.use, activando las señales de StallF, StallD y FlushE para evitar el uso de información que aún no estuviera disponible. Estos resultados nos demostraron que el módulo es capaz de preservar adecuadamente la ejecución del programa, minimizando los problemas de desempeño.

El testbench de PipeReg_EX_MEM nos ayudó a simular el registro del pipeline entre las etapas de EXECUTE y Memory, y comprobó que las señales de control y los datos generados durante EXECUTE son almacenados y transmitidos correctamente hacia MEM en cada flanco ascendente del reloj. Aparte pudimos notar que al activarse el reset, todas las salidas del registro adoptan valores nulos, garantizando un estado inicial predeterminado. En general, nos ayudó a ver que el sistema cumpla adecuadamente la sincronización, permitiendo que varias instrucciones avancen simultáneamente sin interferir una entre otra.

El testbench de PipeReg_ID_EX nos permitió analizar entre las etapas de decodificación y ejecución para poder comprobar la correcta transmisión de señales de control, operandos, datos inmediatos y direcciones de registros hacia la etapa EXECUTE. Aparte pudimos verificar la activación de la señal CLR, que provoca que todas las salidas se despejen, lo que equivale a una instrucción nula dentro del pipeline. Esta funcionalidad es vital para manejar riesgos detectados por la unidad de Hazard, ya que asegura que el procesador mantenga un buen comportamiento frente a dependencias de datos o cambios en el flujo.



En cuanto al testbench de PipeReg_IF_ID , nos sirvió para verificar entre las etapas de FETCH y DECODE que la instrucción sacada de la memoria, así como el valor de

program counter fueran almacenados correctamente cuando la señal de habilitación está activa. Aparte pudimos comprobar que CLR y reset reinician adecuadamente el contenido del registro, evitando que se propaguen instrucciones incorrectas. Esto es muy importante ya que nos permite detener el pipeline mientras se resuelven dependencias y descartar instrucciones especulativas cuando se modifica el flujo de ejecución.

A continuación, está el testbench realizado para PipeReg_MEM_WB , que usamos para ver entre las etapas de MEM y WriteBack que los datos provenientes de la memoria, los resultados de la ALU y las señales de control fueran transferidas correctamente hacia la etapa de WB. De igual manera, el módulo respondió bien a la señal de reset, poniendo todas sus salidas en 0. Esto nos garantizó que sólo los resultados válidos alcanzarán la etapa final del pipeline.

Finalmente, está el test bench del módulo de nivel superior, el top, que nos permitió evaluar el funcionamiento integrado del sistema completo. Gracias a este pudimos observar el avance simultáneo de múltiples instrucciones en las diferentes etapas del pipeline, así como la interacción entre la unidad de control, los registros entre cada etapa, la hazard unit y el resto de bloques que componen el sistema. Los resultados de este nos indican que la arquitectura opera de manera adecuada y según lo esperado, ejecutando de manera correcta las instrucciones y resolviendo de buena manera las situaciones de riesgo que hubo durante su ejecución.

Timing Analyzer

Parallel Compilation		
<<Filter>>		
	Processors	Number
1	Number detected on machine	20
2	Maximum allowed	14
3		
4	Average used	1.06
5	Maximum used	14
6		
7	▼ Usage by Processor	% Time Used
1	Processor 1	100.0%
2	Processors 2-14	0.5%

Timing Analyzer Summary	
<<Filter>>	
Quartus Prime Version	Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Timing Analyzer	Legacy Timing Analyzer
Revision Name	riscvpipeline
Device Family	MAX 10
Device Name	10M50DAF484C7G
Timing Models	Final
Delay Model	Combined
Rise/Fall Delays	Enabled



	Module Name	Elapsed Time	Average Processors Used	Peak Virtual Memory	Total CPU Time (on all processors)
1	Analysis & Synthesis	00:00:14	1.0	4823 MB	00:00:21
2	Fitter	00:00:11	1.0	6515 MB	00:00:15
3	Assembler	00:00:02	1.0	4839 MB	00:00:03
4	Timing Analyzer	00:00:04	1.1	4938 MB	00:00:03
5	EDA Netlist Writer	00:00:01	1.0	4686 MB	00:00:01
6	Total	00:00:32	--	--	00:00:43

El análisis temporal sirvió para poder comprobar si el sistema cumplía o no con los requerimientos temporales de la FPGA con la que lo estaríamos probando, lo cuál validaba que la segmentación contribuye a incrementar la frecuencia de operación respecto a una arquitectura single cycle.

Conclusión

El procesador RISC-V implementado logró de forma exitosa ejecutar las instrucciones usando una arquitectura segmentada de 5 etapas, esto respaldado por las simulaciones individuales realizadas para comprobar el comportamiento de cada módulo, mientras que la simulación del sistema completo demostró el buen funcionamiento de las etapas del pipeline.

Por otro lado, es importante mencionar que la unidad de hazards fue útil para poder resolver las dependencias existentes entre el forwarding y la inserción de los stall, logrado evitar errores en la ejecución y mejorando el desempeño del sistema. Aparte, cabe destacar que los registros entre cada etapa garantizaron la sincronización y el aislamiento adecuado entre fases.

Bibliografía

Harris, S. L., & Harris, D. M. (2022). Digital design and computer architecture (RISC-V edition). Elsevier Inc. <https://documentcloud.adobe.com/spointegration/index.html#>